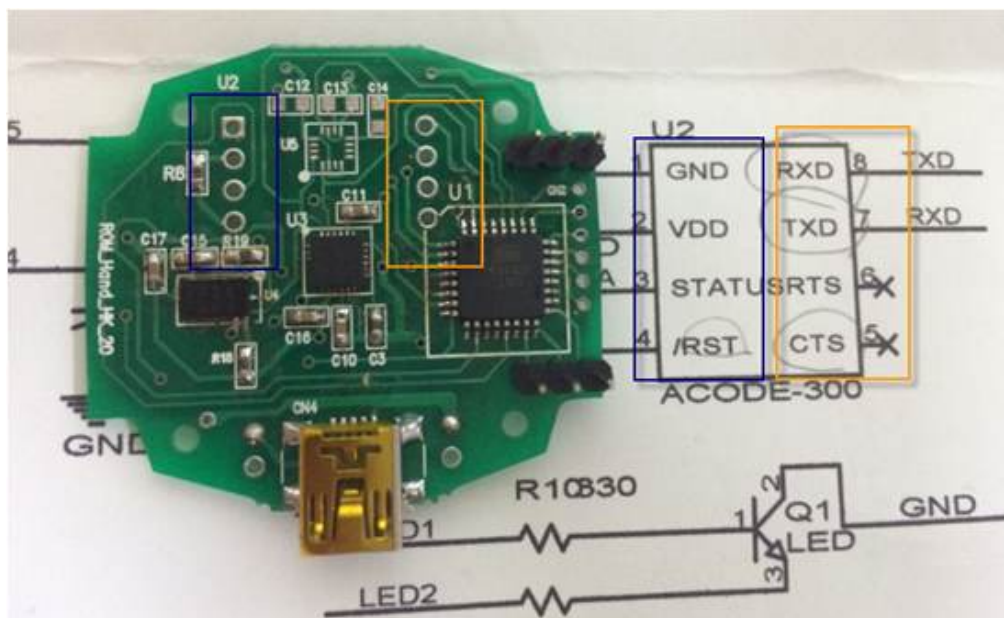
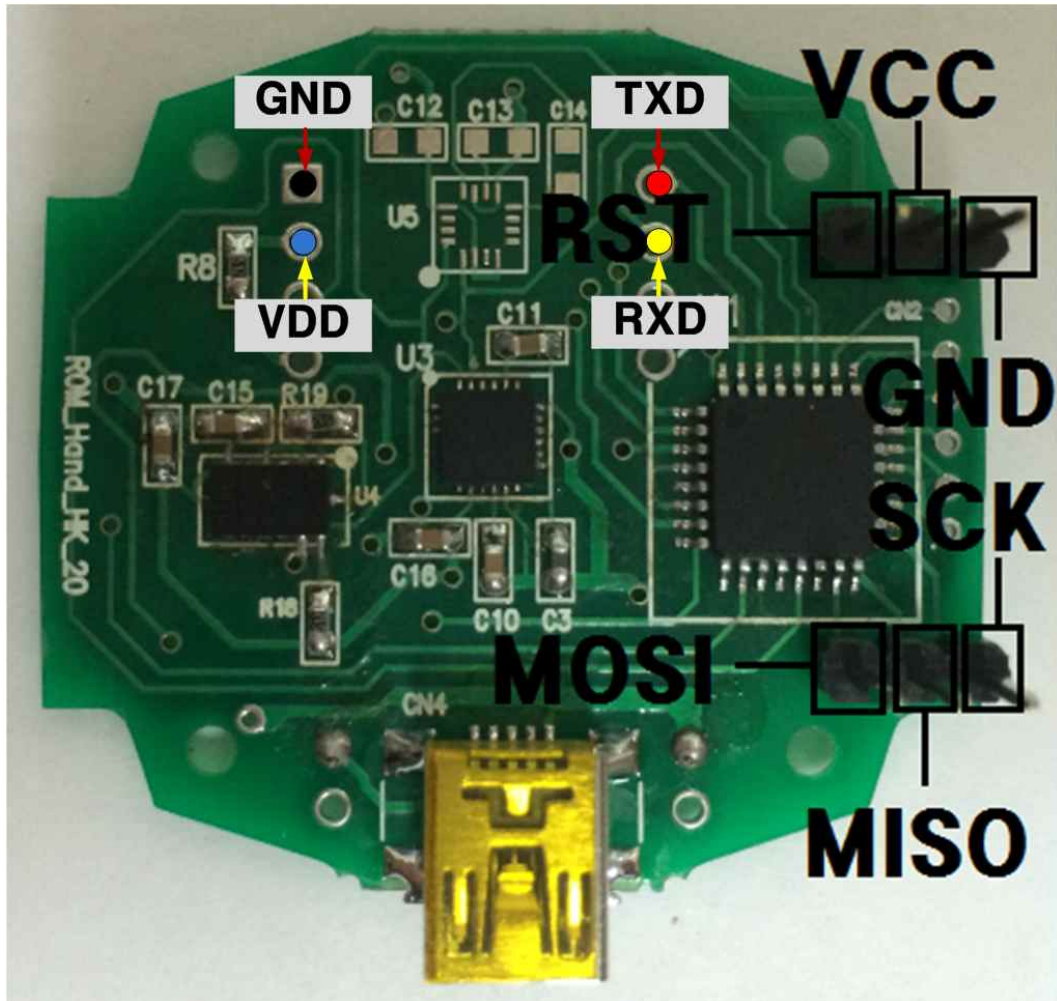


I2C통신을 이용한 자이로 센서와 가속도 센서 데이터 취득 프로그램

2014년 5월 20일 작성자 : 이대중

○ 기본 모듈



목 차

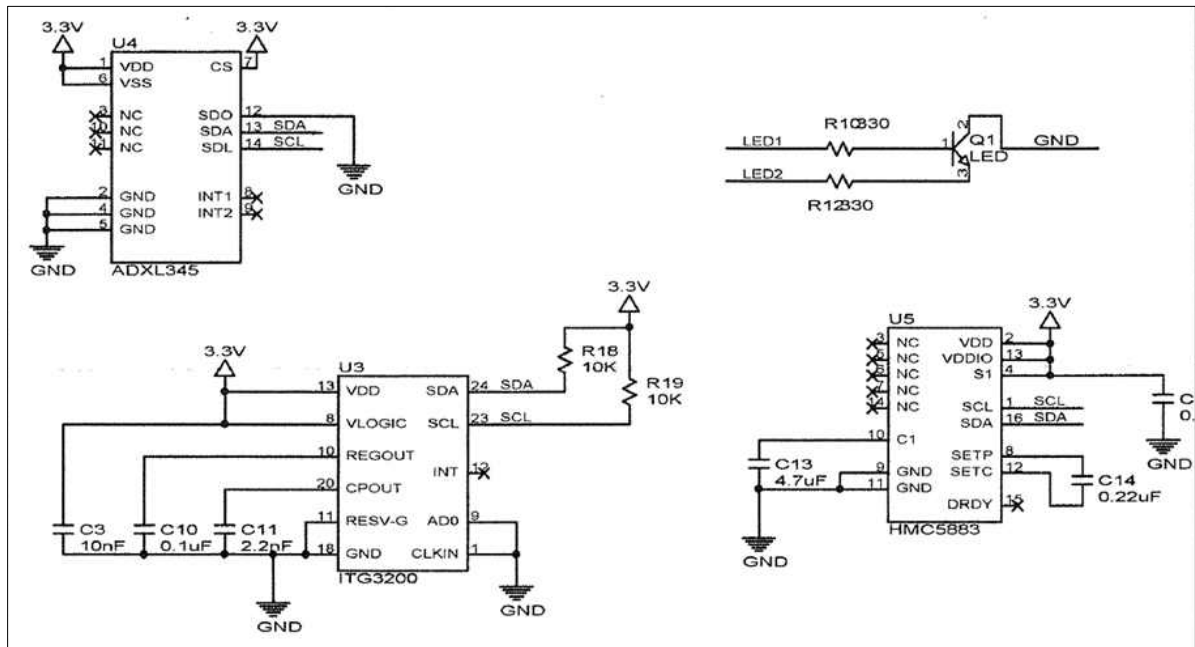
1. 기본 회로도	3
2. ADXL 테스트	5
3. ITG3200 테스트	11
4. ADXL345와 ITG3200 테스트	17
5. ADXL345와 ITG3200 최종 프로그램	19

[부록-1] 통신 프로토타입

[부록-2] ADXL345 데이터 시트

[부록-3] GYRO3200 데이터시트

- 센서부



HardWare setting for ITG3200

- 10kOhm pull-ups on I2C lines.
- VDD & VLOGIC = 3.3V
- SDA -> PC4
- SCL -> PC5
- AD0 = GND;
- CLK -> GND

HardWare setting for ADXL345

- VDD & VSS & CS = 3.3V
- SDC = GND
- SDA -> PC4
- SCL -> PC5

※ 모듈 연결할 때 주의 할 점

1. 5V인 ISP 다운로드 케이블에서 전원을 인가하지 말 것 (USB 전원 단자에서 공급해 줌)
⇒ 이 경우 센서에 3.3V 전원이 아닌 5V 전원이 인가됨
2. 기본 모듈에서 RX 단자와 TX 단자 확인
3. 프로그램 MCU에 올릴 때 스위치 누르고 있을 것

2. ADXL 테스트

- 폴더명 : ADXL_TEST

○ ADXL 레지스터 맵 구성

REGISTER MAP

Table 16. Register Map

Address		Name	Type	Reset Value	Description
Hex	Dec				
0x00	0	DEVID	R	11100101	Device ID.
0x01 to 0x01C	1 to 28	Reserved			Reserved. Do not access.
0x1D	29	THRESH_TAP	R/W	00000000	Tap threshold.
0x1E	30	OFSX	R/W	00000000	X-axis offset.
0x1F	31	OFSY	R/W	00000000	Y-axis offset.
0x20	32	OFSZ	R/W	00000000	Z-axis offset.
0x21	33	DUR	R/W	00000000	Tap duration.
0x22	34	Latent	R/W	00000000	Tap latency.
0x23	35	Window	R/W	00000000	Tap window.
0x24	36	THRESH_ACT	R/W	00000000	Activity threshold.
0x25	37	THRESH_INACT	R/W	00000000	Inactivity threshold.
0x26	38	TIME_INACT	R/W	00000000	Inactivity time.
0x27	39	ACT_INACT_CTL	R/W	00000000	Axis enable control for activity and inactivity detection.
0x28	40	THRESH_FF	R/W	00000000	Free-fall threshold.
0x29	41	TIME_FF	R/W	00000000	Free-fall time.
0x2A	42	TAP_AXES	R/W	00000000	Axis control for tap/double tap.
0x2B	43	ACT_TAP_STATUS	R	00000000	Source of tap/double tap.
0x2C	44	BW_RATE	R/W	00001010	Data rate and power mode control.
0x2D	45	POWER_CTL	R/W	00000000	Power-saving features control.
0x2E	46	INT_ENABLE	R/W	00000000	Interrupt enable control.
0x2F	47	INT_MAP	R/W	00000000	Interrupt mapping control.
0x30	48	INT_SOURCE	R	00000010	Source of interrupts.
0x31	49	DATA_FORMAT	R/W	00000000	Data format control.
0x32	50	DATA0	R	00000000	X-Axis Data 0.
0x33	51	DATA1	R	00000000	X-Axis Data 1.
0x34	52	DATAY0	R	00000000	Y-Axis Data 0.
0x35	53	DATAY1	R	00000000	Y-Axis Data 1.
0x36	54	DATAZ0	R	00000000	Z-Axis Data 0.
0x37	55	DATAZ1	R	00000000	Z-Axis Data 1.
0x38	56	FIFO_CTL	R/W	00000000	FIFO control.
0x39	57	FIFO_STATUS	R	00000000	FIFO status.

○ 메인 프로그램 (AVRSTUDIO 4.18 버전 이용, RS232 통신속도 9600고정)

```
#include<avr/io.h>

#define F_CPU 8000000
#include <util/delay.h>
#include <stdio.h>

#include "i2c.h"
#include "ADXL_345.h"

float adxlx_angle = 0, adxly_angle = 0, adxlz_angle = 0;

int UART0_PutChar(char c, FILE *unused);
void UART0_Init();

int main(void)
{
    i2cInit(); _delay_ms(10);

    UART0_Init();    ADXL345_INIT(); _delay_ms(100);

    while(1)
    {
        ADXL345_Angle(&adxlx_angle, &adxly_angle, &adxlz_angle);
        printf("X: %5.2f,    Y: %5.2f,    Z: %5.2f\r\n", adxlx_angle, adxly_angle, adxlz_angle);
        _delay_ms(500);
    }
}

int UART0_PutChar(char c, FILE *unused)
{
    register unsigned int i = 0;

    while (((UCSR0A>>UDRE0) & 0x01) == 0x00) {
        if(i++ > 10000) break;
    }
    UDR0 = c;
    return (1);
}

void UART0_Init()
{
    UBRR0H = 0;    UBRR0L =51;        // Baud rate
    UCSR0A = 0x00; UCSR0B = 0x18;    // Rx/Tx enable, 8 data,
    UCSR0C = 0x06;    // 비동기식, 패리티금지, 스탑비트1개, 8비트데이터전송

    fdevopen(UART0_PutChar, NULL);    // Set UART0 for printf
}
```

[부함수 설명]

○ i2cInit(); // I2C 통신속도 100KHZ 설정

<pre>void i2cInit(void) { cbi(TWSR, TWPS0); cbi(TWSR, TWPS1); TWBR = 32; //SCL : 100KHZ }</pre>	SCL주파수 : $\frac{8MHz}{16 + 2(32) \times 4^0} = 100KHz$
---	---

○ ADXL345_INIT(); //ADXL345 레지스터 초기화

<pre>void ADXL345_INIT(void) { // stop measuring i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_POWER_CTL, ADXL345_VAR_STOP); //① //Defalut 값으로 셋팅 ADXL345_REG.THRESH_TAP = 0; // Tap threshold ADXL345_REG.OFSX = 0; ADXL345_REG.OFSY = 0; ADXL345_REG.OFSZ = 0; ADXL345_REG.DUR = 0; ADXL345_REG.Latent = 0; ADXL345_REG.Window = 0; ADXL345_REG.THRESH_ACT = 0; ADXL345_REG.THRESH_INACT = 0; ADXL345_REG.TIME_INACT = 0; ADXL345_REG.BW_RATE.all = 0b00001010; ADXL345_REG.INT_SOURCE.all = 0b00000010; ADXL345_REG.DATA_FORMAT.bit.Justify = 0; ADXL345_Write(); //② }</pre>
--

① ADXL 측정 정지 모드로 초기 설정

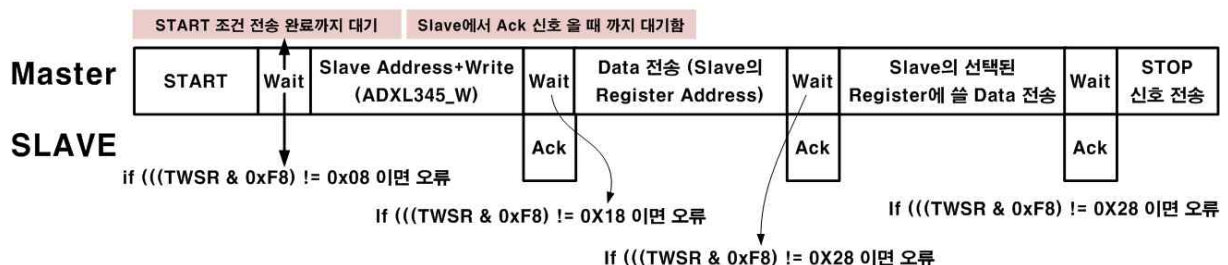
Register 0x2D—POWER_CTL (Read/Write)

D7	D6	D5	D4	D3	D2	D1	D0
0	0	Link	AUTO_SLEEP	Measure	Sleep	Wakeup	

[참고] ADXL의 SINGLEBYTE-WIRTE I2C 형식

- Device address : 0xA6 for a write and 0xA7 for a read.

SINGLE-BYTE WREIT 형식



② ADXL345 레지스터 셋팅 및 측정 모드로 변환

```
void ADXL345_Write(void)
{
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_THRESH_TAP, ADXL345_REG.THRESH_TAP);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_OFSX, ADXL345_REG.OFSX);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_OFSY, ADXL345_REG.OFSY);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_OFSZ, ADXL345_REG.OFSZ);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_DUR, ADXL345_REG.DUR);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_LATENT, ADXL345_REG.Latent);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_WINDOW, ADXL345_REG.Window);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_THRESH_ACT, ADXL345_REG.THRESH_ACT);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_THRESH_INACT, ADXL345_REG.THRESH_INACT);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_TIME_INACT, ADXL345_REG.TIME_INACT);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_ACT_INACT_CTL, ADXL345_REG.ACT_INACT_CTL.all);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_THRESH_FF, ADXL345_REG.THRESH_FF);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_TIME_FF, ADXL345_REG.TIME_FF);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_TAP_AXES, ADXL345_REG.TAP_AXES.all);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_BW_RATE, ADXL345_REG.BW_RATE.all);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_POWER_CTL, ADXL345_REG.POWER_CTL.all);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_INT_ENABLE, ADXL345_REG.INT_ENABLE.all);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_INT_MAP, ADXL345_REG.INT_MAP.all);
    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_DATA_FORMAT, ADXL345_REG.DATA_FORMAT.all);

    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_FIFO_CTL, ADXL345_REG.FIFO_CTL.all);

    i2c_SingleByteWrite(ADXL345_ADDRESS, ADXL345_POWER_CTL, ADXL345_VAR_START); // start measuring

    // write check
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_POWER_CTL, &ADXL345_REG.POWER_CTL.all, 1);
    if(ADXL345_REG.POWER_CTL.all != ADXL345_VAR_START) ADXL345_Flag = 0;
    else ADXL345_Flag = 1;
}
```

○ ADX1345_Angle () 함수에서 각도 계산

```
unsigned char ADX1345_Angle(float* X_ang, float* Y_ang, float* Z_ang)
{
    int X, Y, Z;
    unsigned char status;

    status = ADX1345_ReadXYZ(&X, &Y, &Z);

    *X_ang = atan((float)X/sqrt((float)Y*Y + (float)Z*Z))*57.2958;
    *Y_ang = atan((float)Y/sqrt((float)X*X + (float)Z*Z))*57.2958;
    *Z_ang = atan((float)Z/sqrt((float)X*X + (float)Y*Y))*57.2958;

    return status;
}
```

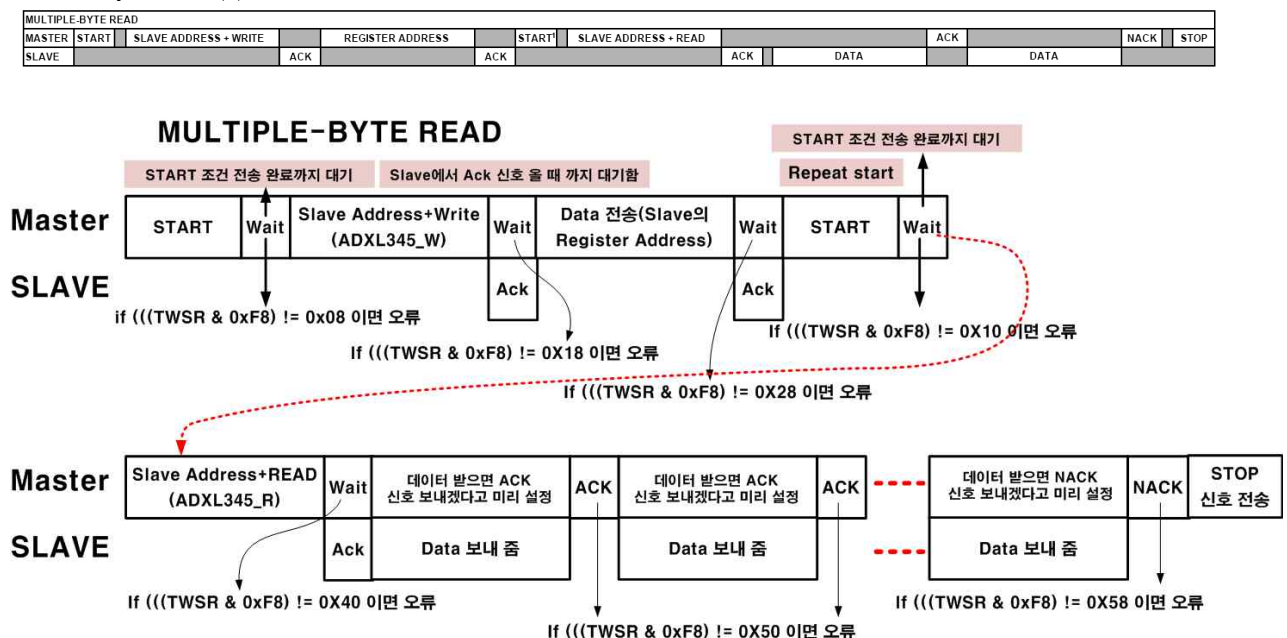
```
unsigned char ADX1345_ReadXYZ(int* X, int* Y, int* Z)
{
    unsigned char ucTemp[6] = {0x00, };
    unsigned char status;

    status = i2c_ByteRead(ADXL345_ADDRESS, ADXL345_DATA0, ucTemp, 6);

    *X = ucTemp[0] | (ucTemp[1] << 8);
    *Y = ucTemp[2] | (ucTemp[3] << 8);
    *Z = ucTemp[4] | (ucTemp[5] << 8);

    return status;
}
```

※ i2c_ByteRead() 함수에서 MULTIPLE-BYTE READ 형식으로 읽어 옴



[참고] ADXL345_ReadAll() 함수를 통하여 현재 설정된 레지스터 값 볼 수 있음

```
void ADXL345_ReadAll(void)
{
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_DEVID, &ADXL345_REG.DEVID, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_THRESH_TAP, &ADXL345_REG.THRESH_TAP, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_OFSX, &ADXL345_REG.OFSX, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_OFSY, &ADXL345_REG.OFSY, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_OFSZ, &ADXL345_REG.OFSZ, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_DUR, &ADXL345_REG.DUR, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_LATENT, &ADXL345_REG.Latent, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_WINDOW, &ADXL345_REG.Window, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_THRESH_ACT, &ADXL345_REG.THRESH_ACT, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_THRESH_INACT, &ADXL345_REG.THRESH_INACT, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_TIME_INACT, &ADXL345_REG.THRESH_INACT, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_ACT_INACT_CTL, &ADXL345_REG.ACT_INACT_CTL.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_THRESH_FF, &ADXL345_REG.THRESH_FF, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_TIME_FF, &ADXL345_REG.TIME_FF, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_TAP_AXES, &ADXL345_REG.TAP_AXES.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_ACT_TAP_STATUS, &ADXL345_REG.ACT_TAP_STATUS.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_BW_RATE, &ADXL345_REG.BW_RATE.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_POWER_CTL, &ADXL345_REG.POWER_CTL.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_INT_ENABLE, &ADXL345_REG.INT_ENABLE.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_INT_MAP, &ADXL345_REG.INT_MAP.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_INT_SOURCE, &ADXL345_REG.INT_SOURCE.all, 1);
    i2c_ByteRead(ADXL345_ADDRESS, ADXL345_DATA_FORMAT, &ADXL345_REG.DATA_FORMAT.all, 1);

    printf("ADXL345_DEVID: 0X%02X\r\n", ADXL345_REG.DEVID);
    printf("ADXL345_THRESH_TAP: 0X%02X\r\n", ADXL345_REG.THRESH_TAP);
    printf("ADXL345_OFSX: 0X%02X\r\n", ADXL345_REG.OFSX);
    printf("ADXL345_OFSY: 0X%02X\r\n", ADXL345_REG.OFSY);
    printf("ADXL345_OFSZ: 0X%02X\r\n", ADXL345_REG.OFSZ);
    printf("ADXL345_DUR: 0X%02X\r\n", ADXL345_REG.DUR);
    printf("ADXL345_LATENT: 0X%02X\r\n", ADXL345_REG.Latent);
    printf("ADXL345_WINDOW: 0X%02X\r\n", ADXL345_REG.Window);
    printf("ADXL345_THRESH_ACT: 0X%02X\r\n", ADXL345_REG.THRESH_ACT);
    printf("ADXL345_THRESH_INACT: 0X%02X\r\n", ADXL345_REG.THRESH_INACT);
    printf("ADXL345_TIME_INACT: 0X%02X\r\n", ADXL345_REG.TIME_INACT);
    printf("ADXL345_ACT_INACT_CTL: 0X%02X\r\n", ADXL345_REG.ACT_INACT_CTL.all);
    printf("ADXL345_THRESH_FF: 0X%02X\r\n", ADXL345_REG.THRESH_FF);
    printf("ADXL345_TIME_FF: 0X%02X\r\n", ADXL345_REG.TIME_FF);
    printf("ADXL345_TAP_AXES: 0X%02X\r\n", ADXL345_REG.TAP_AXES.all);
    printf("ADXL345_ACT_TAP_STATUS: 0X%02X\r\n", ADXL345_REG.ACT_TAP_STATUS.all);
    printf("ADXL345_BW_RATE: 0X%02X\r\n", ADXL345_REG.BW_RATE.all);
    printf("ADXL345_POWER_CTL: 0X%02X\r\n", ADXL345_REG.POWER_CTL.all);
    printf("ADXL345_INT_ENABLE: 0X%02X\r\n", ADXL345_REG.INT_ENABLE.all);
    printf("ADXL345_INT_MAP: 0X%02X\r\n", ADXL345_REG.INT_MAP.all);
    printf("ADXL345_INT_SOURCE: 0X%02X\r\n", ADXL345_REG.INT_SOURCE.all);
    printf("ADXL345_DATA_FORMAT: 0X%02X\r\n", ADXL345_REG.DATA_FORMAT.all);
}
```

3. ITG3200 테스트 [폴더명 : ITG_TEST]

○ ITG_3200 레지스터 맵 구성

Addr Hex	Addr Decimal	Register Name	R/W	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
0	0	WHO_AM_I	R/W	0	ID						-
15	21	SMPLRT_DIV	R/W	SMPLRT_DIV							
16	22	DLPF_FS	R/W	-	-	-	FS_SEL		DLPF_CFG		
17	23	INT_CFG	R/W	ACTL	OPEN	LATCH_INT_EN	INT_ANYRD_2CLEAR	-	ITG_RDY_EN	-	RAW_RDY_EN
1A	26	INT_STATUS	R	-	-	-	-	-	ITG_RDY	-	RAW_DATA_RDY
1B	27	TEMP_OUT_H	R	TEMP_OUT_H							
1C	28	TEMP_OUT_L	R	TEMP_OUT_L							
1D	29	GYRO_XOUT_H	R	GYRO_XOUT_H							
1E	30	GYRO_XOUT_L	R	GYRO_XOUT_L							
1F	31	GYRO_YOUT_H	R	GYRO_YOUT_H							
20	32	GYRO_YOUT_L	R	GYRO_YOUT_L							
21	33	GYRO_ZOUT_H	R	GYRO_ZOUT_H							
22	34	GYRO_ZOUT_L	R	GYRO_ZOUT_L							
3E	62	PWR_MGM	R/W	H_RESET	SLEEP	STBY_XG	STBY_YG	STBY_ZG	CLK_SEL		

○ 메인 프로그램 (AVRSTUDIO 4.18 버전 이용, RS232 통신속도 9600고정)

```
#include<avr/io.h>

#define F_CPU 8000000
#include <util/delay.h>
#include <stdio.h>

#include "i2c.h"
#include "ITG_3200.h"

int gyro_x = 0, gyro_y = 0, gyro_z = 0;

int UART0_PutChar(char c, FILE *unused);
void UART0_Init();

int main(void)
{
    i2cInit(); _delay_ms(10);

    UART0_Init();    ITG3200_INIT();    _delay_ms(100);

    while(1)
    {
        ITG3200_ReadXYZ(&gyro_x, &gyro_y, &gyro_z);
        printf("X: %+06d,    Y: %+06d,    Z: %+06d\r\n", gyro_x, gyro_y, gyro_z);
        _delay_ms(500);
    }
}

int UART0_PutChar(char c, FILE *unused) { //생략 }
void UART0_Init() { //생략 }
```

[부함수 설명]

○ **ITG3200_INIT()**; //ITG3200 레지스터 초기화

ITG3200_ADDRESS는 0xD0(Write), 0xD1(Read)

```
volatile unsigned char ITG3200_Flag = 0;

void ITG3200_INIT()
{
    unsigned char ucTemp = 0;

    // Reset to defaults
    i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_PWR_M, ITG3200_VAR_RESET); //①
    _delay_ms(1);

    // SMLPRT_DIV = 0
    i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_SMPL, ITG3200_VAR_SMLPRT); //②

    // DLPF_CFG = 0, FS_SEL = 3
    i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_DLPF, ITG3200_VAR_DLPF); //③

    i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_INT_C, ITG3200_VAR_INT); //④
    i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_PWR_M, ITG3200_VAR_PWON); //⑤

    // write check
    i2c_ByteRead(ITG3200_ADDRESS, ITG3200_REG_SMPL, &ucTemp, 1);
    if(ucTemp != ITG3200_VAR_SMLPRT) return;

    i2c_ByteRead(ITG3200_ADDRESS, ITG3200_REG_DLPF, &ucTemp, 1);
    if(ucTemp != ITG3200_VAR_DLPF) return;

    i2c_ByteRead(ITG3200_ADDRESS, ITG3200_REG_INT_C, &ucTemp, 1);
    if(ucTemp != ITG3200_VAR_INT) return;

    i2c_ByteRead(ITG3200_ADDRESS, ITG3200_REG_PWR_M, &ucTemp, 1);
    if(ucTemp != ITG3200_VAR_PWON) return;

    ITG3200_Flag = 1;
}
```

① **i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_PWR_M, ITG3200_VAR_RESET);**

⇒ ITG3200_VAR_RESET 0x80

※ Register 62 - Power Management (초기 Reset 시킴)

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Default Value
3E	62	H_RESET	SLEEP	STBY_XG	STBY_YG	STBY_ZG	CLK_SEL			00h

② `i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_SMPL, ITG3200_VAR_SMLPRT);`
 $\Rightarrow$ ITG3200_VAR_SMLPRT 0x00

※ Register 21 - Sample Rate Divider

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Default Value
15	21	SMPLRT_DIV								00h

- ITG-3200의 샘플링 레이트 결정
 $\Rightarrow F_{\text{sample}} = F_{\text{internal}} / (\text{divider} + 1),$
 $\Rightarrow$ SMPLRT_DIV Sample rate divider: 0 to 255

③ `i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_DLPF, ITG3200_VAR_DLPF);`
 $\Rightarrow$ ITG3200_VAR_DLPF 0x18

※ Register 22 - DLPF, Full Scale ($\pm 2000^\circ$, 샘플링 레이트 8KHZ 선택)

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Default Value
16	22	-			FS_SEL		DLPF_CFG			00h

FS_SEL

FS_SEL	Gyro Full-Scale Range
0	Reserved
1	Reserved
2	Reserved
3	$\pm 2000^\circ/\text{sec}$

DLPF_CFG

DLPF_CFG	Low Pass Filter Bandwidth	Internal Sample Rate
0	256Hz	8kHz
1	188Hz	1kHz
2	98Hz	1kHz
3	42Hz	1kHz
4	20Hz	1kHz
5	10Hz	1kHz
6	5Hz	1kHz
7	Reserved	Reserved

④ `i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_INT_C, ITG3200_VAR_INT);`
 $\Rightarrow$ ITG3200_VAR_INT 0x05

※ Register 23 - Interrupt Configuration (장치의 인터럽트 동작을 구성함)

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Default Value
17	23	ACTL	OPEN	LATCH_INT_EN	INT_ANYRD_2CLEAR	0	ITG_RDY_EN	0	RAW_RDY_EN	00h

- ACTL Logic level for INT output pin - 1=active low, 0=active high
- OPEN Drive type for INT output pin - 1=open drain, 0=push-pull
- LATCH_INT_EN Latch mode - 1=latch until interrupt is cleared, 0=50us pulse
- INT_ANYRD_2CLEAR Latch clear method - 1=any register read, 0=status register read only
- ITG_RDY_EN Enable interrupt when device is ready (PLL ready after changing clock source)
- RAW_RDY_EN Enable interrupt when data is available
- 0 Load zeros into Bits 1 and 3 of the Interrupt Configuration register.

⑤ `i2c_SingleByteWrite(ITG3200_ADDRESS, ITG3200_REG_PWR_M, ITG3200_VAR_PWON);`

※ Register 62 - Power Management (파워 온 시킴)

$\Rightarrow$ ITG3200_VAR_PWON 0x00

Type: Read/Write

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Default Value
3E	62	H_RESET	SLEEP	STBY_XG	STBY_YG	STBY_ZG	CLK_SEL			00h

○ **ITG3200_ReadXYZ(&gyrox, &gyroy, &gyroz);**

ITG3200_ADDRESS는 0xD0(Write), 0xD1(Read)

```
#define ITG3200_REG_GX_H    0x1D
#define ITG3200_REG_GX_L    0x1E
#define ITG3200_REG_GY_H    0x1F
#define ITG3200_REG_GY_L    0x20
#define ITG3200_REG_GZ_H    0x21
#define ITG3200_REG_GZ_L    0x22
```

```
unsigned char ITG3200_ReadXYZ(int* X, int* Y, int* Z)
{
    unsigned char temp = 0;
    unsigned char Value[6] = {0x00, };
    unsigned char status = 0;
    register unsigned char i;
    int gyrox = 0, gyroy = 0, gyroz = 0;

    for (i = 0; i < ITG3200_NUMOFSAMPLE; i++)
    {
        do {
            status = i2c_ByteRead(ITG3200_ADDRESS, ITG3200_REG_INT_S, temp, 1);
        } while (!(temp & 0x01) && (status == 1));

        status = i2c_ByteRead(ITG3200_ADDRESS, ITG3200_REG_GX_H, Value, 6);

        gyrox += (Value[0] << 8) | Value[1];
        gyroy += (Value[2] << 8) | Value[3];
        gyroz += (Value[4] << 8) | Value[5];
    }
    *X = gyrox/ITG3200_NUMOFSAMPLE;
    *Y = gyroy/ITG3200_NUMOFSAMPLE;
    *Z = gyroz/ITG3200_NUMOFSAMPLE;

    return status;
}
```

Register 26 – Interrupt Status

#define ITG3200_REG_INT_S 0x1A

Type: Read only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0	Default Value
1A	26	-	-	-	-	-	ITG_RDY	-	RAW_DATA_RDY	00h

※ RAW_DATA_RDY Raw data is ready

○ ITG3200 정지시 출력값 비교

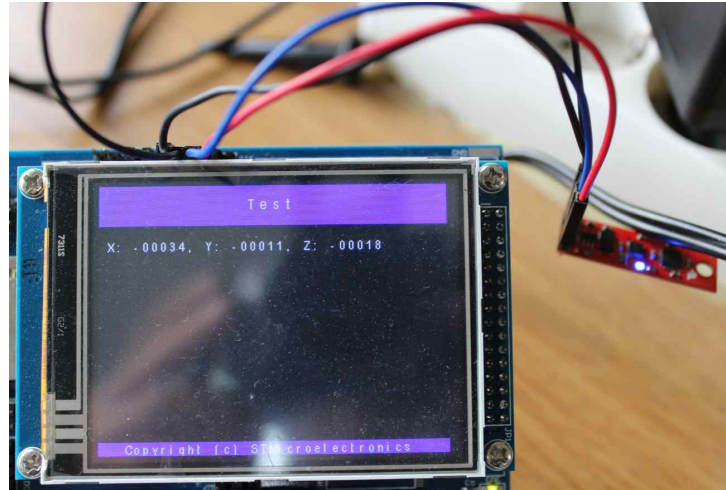
테스트 제품			모듈		
-73	-13	13	-34	-11	-18

```

X: -00073, Y: -00014, Z: +00013
X: -00073, Y: -00013, Z: +00014
X: -00071, Y: -00013, Z: +00014
X: -00073, Y: -00014, Z: +00013
X: -00074, Y: -00013, Z: +00013
X: -00073, Y: -00014, Z: +00015
X: -00072, Y: -00013, Z: +00013
X: -00073, Y: -00013, Z: +00013
X: -00073, Y: -00012, Z: +00012
X: -00073, Y: -00012, Z: +00014
X: -00073, Y: -00014, Z: +00014
X: -00071, Y: -00013, Z: +00013
X: -00073, Y: -00014, Z: +00014
X: -00073, Y: -00014, Z: +00013
X: -00072, Y: -00014, Z: +00014

```

Serial port COM8 closed



4. ADXL345와 ITG3200 테스트 [폴더명 : ADXL_ITG_TEST]

```
#include<avr/io.h>

#define F_CPU 8000000
#include <util/delay.h>
#include <stdio.h>

#include "i2c.h"
#include "ADXL_345.h"
#include "ITG_3200.h"

int UART0_PutChar(char c, FILE *unused);
void UART0_Init();

float  adxlx_angle = 0, adxly_angle = 0, adxlz_angle = 0;
int  gyrox = 0, gyroy = 0, gyroz = 0;

int main(void)
{

    i2cInit();
    _delay_ms(10);

    UART0_Init();
    ADXL345_INIT();
    ITG3200_INIT();

    _delay_ms(100);

    ADXL345_ReadAll();
    _delay_ms(2000);

    while(1)
    {

        ADXL345_Angle(&adxlx_angle, &adxly_angle, &adxlz_angle);
        printf("ADXL X: %5.2f,    Y: %5.2f,    Z: %5.2f\r\n", adxlx_angle, adxly_angle, adxlz_angle);

        ITG3200_ReadXYZ(&gyrox, &gyroy, &gyroz);
        printf("GYRO X: %+06d,    Y: %+06d,    Z: %+06d\r\n", gyrox, gyroy, gyroz);

        _delay_ms(500);
    }
}
// 다음 페이지 이어서 작성
```

```

int UART0_PutChar(char c, FILE *unused)
{
    register unsigned int i = 0;

    while (((UCSR0A>>UDRE0) & 0x01) == 0x00) {
        if(i++ > 10000) break;
    }

    UDR0 = c;

    return (1);
}

void UART0_Init()
{
    UBRR0H = 0;
    UBRR0L = 51; // Baud rate
    UCSR0A = 0x00; UCSR0B = 0x18; // Rx/Tx enable, 8 data,
    UCSR0C = 0x06; // 비동기식, 패리티금지, 스탑비트1개, 8비트데이터전송

    fdevopen(UART0_PutChar, NULL); // Set UART0 for printf
}

```

5. ADXL345와 ITG3200 최종 프로그램 [폴더명 : ADXL_ITG_FINAL]

○ 실행 결과

- 초기 전원 인가시에는 스위치를 누른 후에 LED가 점등되면 스위치를 OFF 한다.
- 전원 OFF 시에는 LED가 소등될 때까지 스위치를 계속 누른다(약 3초 설정).
스위치가 OFF되면 전원이 차단된다.
- 전원이 인가되면 가속도센서와 자이로 센서로부터 출력된 6개의 값이 통신으로 출력

○ 메인 프로그램 (AVRSTUDIO 4.18 버전 이용, RS232 통신속도 9600)

```
#include "main.h"
#include "eeprom.h"
#include "i2c.h"
#include "serial.h"
#include "ADXL_345.h"
#include "ITG_3200.h"

#include <stdio.h>
#include <string.h>
#include <util/delay.h>
#include <avr/interrupt.h>

//-----
void LED_INIT();
void SW_INIT();
void MAX8877_INIT();

unsigned char    ucBaud_Rate    = 3;
/* Baud rate: 1(2400), 2(4800), 3(9600), 4(14400), 5(19200), 6(28800),
   7(38400), 8(57600), 9(76800), 10(115200) */

unsigned char    ucSend_Flag = 0;          // 0(send stop), 1(send start)
unsigned char    ucSerial_High = 0;        // Serial code high
unsigned char    ucSerial_Low = 0;         // Serial code low
unsigned char    ucCali_X = 0;             // X calibration
unsigned char    ucCali_Y = 0;            // Y calibration
unsigned char    ucCali_Z = 0;            // Z calibration

unsigned char    Timer_Flag = 0;

int gyrox = 0;
int gyroy = 0;
int gyroz = 0;

int adxlx = 0;
int adxly = 0;
int adxlz = 0;

float    adxlx_angle = 0;
float    adxly_angle = 0;
float    adxlz_angle = 0;

//다음 페이지 이어서 작성
```

```

//-----
/* Timer Interrupt Routine */
SIGNAL (TIMER1_COMPA_vect)    // 20Hz
{
    if(ucSend_Flag == 1)
    {
        if(Timer_Flag == 0) Timer_Flag = 1;
    }
}

//-----
int main(void)
{
    volatile unsigned char count = 0;

    LED_INIT();
    SW_INIT();

    MAX8877_INIT();

    i2cInit();
    _delay_ms(10);

    // read data from eeprom
    ucBaud_Rate = EEPROM_Read_char(ADD_BAUD_RATE); // EEPROM Read baud rate
    UART0_Init(ucBaud_Rate); // UART0 initialization

    ucSerial_High = EEPROM_Read_char(ADD_SERIAL_HIGH); // EEPROM Read serial code high
    if(ucSerial_High > 99) ucSerial_High = 0;

    ucSerial_Low = EEPROM_Read_char(ADD_SERIAL_LOW); // EEPROM Read serial code low
    if(ucSerial_Low > 99) ucSerial_Low = 0;

    ucCali_X = EEPROM_Read_char(ADD_CALI_X); // EEPROM Read calibration X
    ucCali_Y = EEPROM_Read_char(ADD_CALI_Y); // EEPROM Read calibration Y
    ucCali_Z = EEPROM_Read_char(ADD_CALI_Z); // EEPROM Read calibration Z

    ADXL345_INIT();
    ITG3200_INIT();

    _delay_ms(100);

    //Timer1: 20Hz
    TIMSK1 |= 1<<OCIE1A;
    TCCR1A = 0x00;
    TCCR1B = 0x0A; // TOP : OCR1A
    TCCR1C = 0x00;
    OCR1A = 50000; // 50ms

    sei(); // enable interrupts

    //다음 페이지 이어서 작성

```

```

while(1)
{
// SW check
if((PINB & 0x40)== 0x40) {
    count++;
    _delay_ms(100);
    if(count >= 25) {
        PORTD = 0x00;
        PORTB &= 0x7F;
        _delay_ms(1);
    }
}
else
{
    count = 0;

    //RS232 process
    if(Get_UART0_Flag() == 1) UART0_Parsing();

    //ADXL345 init check
    if(ADX1345_INIT_Flag() == 0) ADX1345_INIT();

    //ITG3200 init check
    if(ITG3200_INIT_Flag() == 0) ITG3200_INIT();

    if(Timer_Flag == 1)
    {
        ADX1345_Angle(&adxlx_angle, &adxly_angle, &adxlz_angle);
        ITG3200_ReadXYZ(&gyrox, &gyroy, &gyroz);

        printf("SA%04d%04d%04d%06d%06dP\n", 4000+(int)(adxlx_angle*10.0),
            4000+(int)(adxly_angle*10.0), 4000+(int)(adxlz_angle*10.0), gyrox, gyroy, gyroz);
        Timer_Flag = 0;
    }
} //End of if-else
} //End of while
} // End of Main

//-----
void LED_INIT()
{
    DDRD  |= 0b00011000;
    PORTD |= 0b00010000; //LED2 high, LED1 low
}

//-----
void SW_INIT()
{
    DDRB &= 0b10111111;
}

//-----
void MAX8877_INIT()
{
    DDRB  |= 0x80;    PORTB |= 0x80;
    DDRD  |= 0x40;    PORTD |= 0x40;
}

//다음 페이지 이어서 작성

```

```

//-----
void Update_SerialCode(unsigned char ucPos, unsigned char *Buff)
{
    unsigned char ucTemp = ((Buff[3]-0x30) * 10) + (Buff[4]-0x30);

    if((ucTemp < 0) || (ucTemp > 99)) return;    // Bound check

    if(ucPos == 1)                                // Update serial code high
    {
        if(EEPROM_Write_char(ucTemp, ADD_SERIAL_HIGH) == 1)
        {
            UART0_RxReturn();                    // Return Rx
            ucSerial_High = ucTemp;
        }
    }
    else if(ucPos == 0)                            // Update serial code low
    {
        if(EEPROM_Write_char(ucTemp, ADD_SERIAL_LOW) == 1)
        {
            UART0_RxReturn();                    // Return Rx
            ucSerial_Low = ucTemp;
        }
    }
}

//-----
void Update_Send(unsigned char *Buff)
{
    if(Buff[1] == 0x30) ucSend_Flag = 0;        // Stop send
    else if(Buff[1] == 0x31) ucSend_Flag = 1;    // Start send

    UART0_RxReturn();                            // Return Rx
}

//-----
void Send_Serial(void)
{
    printf("SK%02d%02d", ucSerial_High, ucSerial_Low);
}

//-----
void Update_Calibration(unsigned char *Buff)
{
    switch(Buff[3]) {
        case 0x58:                                // X Calibration
            if(EEPROM_Write_char(Buff[4], ADD_CALI_X) == 1)
            {
                UART0_RxReturn();                // Return Rx
                ucCali_X = Buff[4];
            }
            break;

        case 0x59:                                // Y Calibration
            if(EEPROM_Write_char(Buff[4], ADD_CALI_Y) == 1)
            {
                UART0_RxReturn();                // Return Rx
                ucCali_Y = Buff[4];
            }
            break;

        case 0x5A:                                // Z Calibration
            if(EEPROM_Write_char(Buff[4], ADD_CALI_Z) == 1)
            {
                UART0_RxReturn();                // Return Rx
                ucCali_Z = Buff[4];
            }
            break;
    }
}

//다음 페이지 이어서 작성

```

```

//-----
void Update_BaudRate(unsigned char *Buff)
{
    unsigned char ucTemp = ((Buff[3]-0x30) * 100) + ((Buff[4]-0x30) * 10) + (Buff[5]-0x30);

    // Index check
    if((ucTemp < 1) || (ucTemp > 8)) return;
    else
    {
        if(EEPROM_Write_char(ucTemp, ADD_BAUD_RATE) == 1) // EEPROM Write
        {
            UART0_RxReturn(); // Return Rx
            _delay_ms(10);

            ucBaud_Rate = ucTemp;
            UART0_Init(ucBaud_Rate);

            _delay_ms(10);
        }
    }
}

//-----

```